Онтология продукта для команды агентов: памятка

Онтология продукта — файлы в папке `product/` рядом с кодом. В них записано, из чего состоит продукт, как связаны его части и какие решения и требования действуют, со ссылками на код. Онтология лежит в репозитории, её читает агент на любой модели.

Что она даёт

- Агент видит связи, которых нет в тексте кода: куда ведёт экран, какими путями создаётся документ, что решил продакт.
- Агент берёт из онтологии только нужную запись и не тратит токены на чтение всей онтологии.
- Скрипт без участия модели проверяет, выполняет ли изменение агента требования из онтологии.

Где пользы не видно

- Код, который модель уже знает: открытые популярные системы.
- Вопросы, ответ на которые находится поиском по коду.
- Самая дешёвая модель: с онтологией она находит меньше и тратит больше.
- Онтология большой системы, которую агент читает целиком: он тратит больше токенов, чем без неё.

Два слоя

- **Требования и решения** записывают люди с первого дня, сразу в проверяемой форме. Замысел продакта модель из кода не восстановит.
- **Структуру** собирает модель, когда схема данных перестала меняться каждый день. Скрипт сверки находит записи, которые разошлись с кодом.

Что записывать

Требования и решения записывают на любом уровне подробности.

- **Карта:** части продукта и где они лежат в коде.
- **Документы и связи:** сущности, переходы между экранами и документами, все пути создания документа. Это основной уровень, на нём работает проверка изменений.
- **Поля:** откуда поле заполняется, куда переносится, где печатается и выводится в отчётах. Этот уровень дороже в сборке и в чтении. На открытой системе качество он не поднял, на закрытой его пока не проверяли.
- **Функции и вызовы:** граф кода. Его строят инструменты анализа кода, в онтологию он не входит.

Как давать агенту

- **Агенту-разработчику:** оглавление и нужные записи по поиску.
- **Агенту продакта и аналитика:** решения и требования целиком, остальное по запросу.

Требование для агента

- **источник значения:** документ, из которого берётся значение; по нему на ревью видно, что агент подставил своё;
- **исключения** перечислены явно, пустой список значит, что их нет;
- **где действует:** все пути создания документа;
- **открытые вопросы:** пока они есть, изменение этого поля не проходит проверку;
- **тип:** продуктовое нарушение возвращают продакту, техническое исправляет автор.

Решения, которые агент принял сам, люди утверждают на ревью, как код.

Проверка

- `check.py` сверяет онтологию с кодом. Его ставят в CI на каждое изменение.
- `check_change.py` проверяет изменение по требованиям: пути создания, заглушки, ослабление правила. Скрипт видит только то, что записано в онтологии. За полноту путей и за то, откуда берётся значение, отвечает человек на ревью.

Команда агентов

- Модель подбирают по сложности работы: анализ и итоговую проверку делает сильная, код и тексты средняя, проход по чек-листу дешёвая.
- В конце работает ведущий на сильной модели. Он читает отчёт тестировщика и исправляет трудные места.
- Модели сравнивают по цене всей задачи: дорогая за токен модель может потратить на задачу меньше токенов.

С чего начать

С одной части системы, где ошибка дороже всего: документы, деньги, права доступа, обязательные поля. Для существующего продукта подходит та часть, которую меняют сейчас.

Митап «ИИ в разработке ПО», 20.10.2026. Николай Тржаскал, FabricaONE.AI. Скрипты и шаблон — в папке `02_ontology_kit`, модели по ролям — в `03_agent_team`.